〔解　説〕

知識ベースト・プランニング*

中　島　秀　之**

1. はじめに

コンピュータが華々しく登場した当時，人々はそのスピードに多くの夢を託した．機械翻訳がそれであり，また推論エンジンがそれである．前者は元の言語の文と辞書を入力とし，目的言語の文を出力とする．後者は公理を入力とし，定理を出力とする（あるいは公理と定理を入力とし，その定理の証明を出力とする）．残念ながら，これらの夢はことごとく崩れ去った．後に残された結論は，常識（別名，知識）がないと何もできない，ということであった．我々人類が知的にみえるのは，その推論機構が知的であるのではなく，膨大なバックグラウンド知識を持っているからしい．

例えば，機械翻訳システムにおいて

I took a bus to the station.

と

I took a girl to the station.

を訳し分けるには，bus がバスであり，girl が女の子であるという辞書的知識だけでは不十分である．bus が乗りものであり，girl はそうでない，などの知識が必要である．

推論エンジンで用いられる知識は，この例ほど明らかではないが，やはりやみくもに盲サーチをするだけではなく，状況に応じた行動が必要なのは明らかであろう．この状況と行動を結びつけるのが知識であり，その不完全さ故に一般にはヒューリスティクスなどと呼ばれているものである．

本論文では，プランニングという側面から見た知識の表現とその利用に焦点を当ててみたい．第2章では，知識の表現を取りあげる．そして第3章では知識の表現とプランニングに用いうる言語としてPLANNER と Prolog を紹介する．また，トータルシステムの例としてSHRDLU を紹介する．

原稿受付　1984年9月3日

* Knowledge-Based Planning

** 電子技術総合研究所 電子計算機部

2. 知識の表現

我々の持つ知識を計算機内で表現するための枠組はいろいろと提案されてきた．いまのところオールマイティなものはなく，場面に応じて使い分けているようであるが，ここではそれらのうち主だったものを紹介しておこう．また知識表現，プランニングにおいて遭遇する問題についても触れることにする．

2.1 フレーム理論

フレーム理論[6]は主として人間の視覚認識の速さを説明するために提唱された，人間の知識の全体的構成に関する理論である．例えば，友人の家を初めて訪問した場合を考えてみよう．理論的には初めて訪れる家だから，その家の構造に関して何も知らないはずである．極端にいえば見るものすべてが生涯最初の物体であるはずである．にもかかわらず，入口の戸を開いた瞬間に玄関の構造や，そこに存在する数々の物体を認識できるはずである．個々の物体を良く観察していたのでは，とてもそんな高速処理はできない．そこで Minsky は，人間が物事を認識する際にはあらかじめ何が期待されるかを記述したフレーム（枠組み）を持っており，大部分の情報は既にそのフレームに記入されていると考えた．その結果，たとえ初めての家でも玄関の構造に関してはかなりの情報をあらかじめ持っていることになる．実際の認識はそのフレームの細部を埋める手間だけである．

この理論によれば，人間の知識というのは互いに結合しあった多数のフレームからなるフレーム・システムとして存在する．先程の例より少し低レベルの表現としては，物体の視覚情報もこのように蓄えられているとしている．立方体をある一方向から見ていると3つの面しか見えない．しかし，立方体を右に回転するとどんな面が現われてくるかは，別のフレームへのポインタとして存在するので，その情報が次に見えるものの期待として存在する（**図 1**）．

このように，個々のフレームは典型的な状態や知識に対応している．フレームにはスロットとして何種類かの

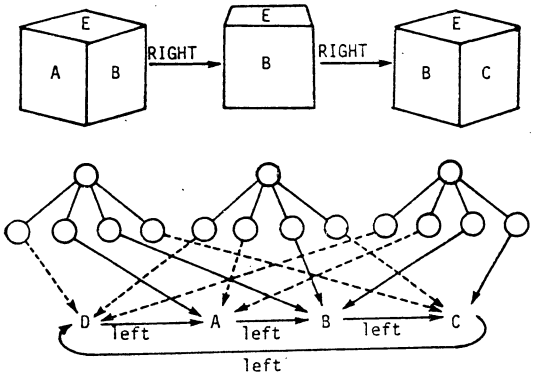


図 1 立方体の様々の視的に対応するフレーム・システム

情報が付加されている．例えば家には屋根や入口，窓などがあるといった情報は，これらに対応するスロットとして蓄えられる．またスロットには，フレーム自身の使い方に関するものや，次に何が起こるかを予測するものや，その予測が裏切られた場合の行動に関するものなどもある．

フレームは典型的な場合に関する知識（プロトタイプと呼ぶ）であるから，個々の事態とは細部が異なるはずである．そこで個々の事態に関してはそれに対応する事例（インスタンス—instance—と呼ぶ）が生成される．これらのインスタンスはプロトタイプフレームに付加された情報を継承することも，しないこともある．例えば，日本の場合なら家の屋根スロットには瓦と書かれているかもしれない．だからといって，すべての家が瓦ぶきとは限らない．このように，普通はこうであるといった情報をデフォールト情報という．他に何も情報が得られない場合にはこのデフォールト情報が用いられるが，これと異なる情報が得られた場合にはデフォールトは直ちに捨て去られる．すなわち，それはインスタンスによって容易に書き換えられる．またあるスロットには情報はなく，インスタンスごとに埋められなければならない．このようなスロットには値に関する制約を記述することもできる．

まとめると，この理論によれば，人間による新しい状況の認識は，その状況にふさわしいプロトタイプフレームを呼出し，そのスロットの詳細を（現実の観察により）埋める作業として記述される．

2.2 意味ネットワーク

意味ネットワーク[10]は最初，人間の記憶の連想性を説明するモデルとして提唱された．人間は，リンゴ・赤・夕日・夕焼け・トンボ…というように関連した事項の想起のスピードが速い．意味ネットワークではこれを，関連した事項をリンクで結合しておき，連想はこのリンクを辿ることによって実現している．

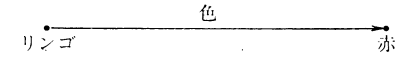


図 2 「リンゴは赤い」を表した意味ネットワーク

意味ネットワークはノードとリンクからなり，ノードは様々の概念，リンクは概念間の関係に対応する．例えば「リンゴは赤い」という知識は図2のように「リンゴ」というノードと「赤」というノードを「色」というリンクで結んで表現する．

意味ネットワークにおいては，ネットワークのつながり方と，そのつながり方を操作し，推論を行うインタープリタの組で知識を表していることになる．これまでのシステムでは，このインタープリタの方は Lisp などの他の言語で書かれることが多かった．ノードやリンクをどのように解釈・操作するかという情報はプログラム中に暗黙のうちに埋め込まれてしまい，なかなか表面には現われてこなかった．

2.3 論 理

我々の持つ知識を論理式で表現することは古くから行われてきた．例えば，

人間は過ちを犯す．

という文は

$$\forall x \text{ 人間}(x) \rightarrow \text{過ちを犯す}(x)$$

のように表現する注1)．

そもそも論理学は人間の思考の定式化を目的として作られたものである．論理的思考というのは，論理式を用いた推論の過程であると考えられる．この推論の規則には様々なものがあるが，例えば三段論法などは有名である．Prolog はこの三段論法を機械的に行っていると考えることもできる[4]．

2.4 プロダクション・システム

プロダクション・システム[8]は問題を解決するための規則の集合からなる．それぞれの規則は原則として互いに独立に記述されるので，大量の規則が存在してもそれらの維持（新しい規則の追加，削除）が容易なのを特徴とする．

各々の規則は，

＜条件＞→＜行動＞

注 1) ここで，∀は「すべての x に関して以下が成立つ」という意味の論理記号，→は「ならば」という意味の論理記号である．論理記号には以下のようなものがある．

∀ すべての (for-all)	∨ または (or)
∃ ある (exists)	→ ならば (implies)
∧ および (and)	¬ 否定 (not)

また，“人間”や“過ちを犯す”を述語記号と呼ぶ．述語記号に引数の続いたものが原子式である．

の形をしており，＜条件＞が成立していれば＜行動＞が起動される．例えば，

金がない→銀行を襲え

という風である．

＜条件＞は現在の状況を反映している短期記憶，STM (Short Term Memory) の状態により決定され，＜行動＞の結果として STM の内容が書替えられる．例えば，銀行の襲撃に成功すれば，STM の金がないという状態は消去され，代わりに大金があるという状態になる．

また，規則自身は長期記憶，LTM (Long Term Memory) に蓄えられている．この STM, LTM の分離は人間の記憶の構造に対応している（と考えられている）．

先に述べたように，規則の起動は STM の内容のみに依存し，規則が他の規則を直接起動することはないので，高いモジュール性が維持できる．すなわち新しい規則の追加や削除は，システム全体の動きを大きく損うことなく達成できる．

プロダクション・システムの条件—行動式のルール記述は人間の考え方にも非常によく馴染むので，エキスパート・システムなどで多く使われている．

2.5 フレーム問題

プランニングを行うときに常に問題となってきたのが，いわゆるフレーム問題である．プランニングは現状を何か別の望ましい状態に変更しようとして行われるものであるから，状態の遷移が不可欠である．ところが古典論理には状態の変化の概念がない．例えば積木の世界で

on(A, B)

すなわち積木Aが積木Bの上にあるという公理があるとすれば，それは未来永劫にわたって成立する．これでは困るので普通，状態という概念を陽に持込んで

on(A, B, S)

のように表現する．すなわち，状態SにおいてはAはBの上にあるという意味である．ところが，こうすると状態間の変化を記述する必要が生じる．例えば状態SでAがBの上にあると，CをDの上からEに移動した後の状態（もはやSではない．普通 [move(C, D, E) ; S] のように表現する）でもやはりAはBの上にあることを推論するための規則

on(A, B, [move(C, D, E) ; S])←on(A, B, S).

あるいは，さらに一般化して，

$\forall x \forall y \forall u \forall v \forall w \forall s$ on(x, y, [move(u, v, w); s])←on(x, y, s)∧ x≠u.

が必要である．n 個の状態と m 個の動作があるとすれば，このような状態・動作間の規則は $n \cdot m$ 個必要である．実際にこれらの間で関係のあるものどうしは，少数であるのに多数の無関係さを記述しなければならないという問題をフレーム問題[5,11]という．これは記述量だけの問題ではなく，推論の量にも関係する．ある状態が成り立つかどうかを知るには，毎回それが最初に定まった時点まで遡らなければならない．

フレーム問題を避けるために，いくつかの対策がとられてきた．いずれも古典論理からの離脱であるが，主に副作用の導入と様相論理の導入の2つに分けられる．STRIPS[2] は前者の例である．STRIPS では全ての動作を状態の変化，すなわち新しい公理の追加と古い公理の削除で記述する．そして変更されなかったものは，そのまま残るので無関係さを記述する必要がない．また実際に公理を書き変えていくのだから，過去に遡った推論をする必要もなくなる．PLANNER や Prolog でも，追加や削除のためのプリミティブを用意して公理を実際に書き変える方法を取っている．Prolog/KR[7] ではさらに，現状を書き変えることなく新しい世界として変化を記述することも可能である．

しかしながら STRIPS の動作に状態変化の記述を付加する手段は，後に不完全であることが判明した．例えばカップとその皿を考えてほしい．カップを移動しても皿はついてこないが，皿を移動すると上のカップもついてくる．この差は「移動」という動作のみで記述することはできない．最近では，こちらのほうをフレーム問題と呼ぶことが多いようである．すなわち，フレーム問題とは，ある動作の影響の及ぶ範囲を厳密に規定するのが困難であることをもいう．

Sandewall は UNLESS という，より一般的なオペレータを提案した[11]．

IS（オブジェクト，性質，状態）

を，“オブジェクト”が“状態”で“性質”をもつという述語，

ends（オブジェクト，性質，状態）

を，“オブジェクト”は“状態”以降では，もはや“性質”をもたないという述語，

Succ（状態，動作）

を“状態”で“動作”を行った次の状態を示す関数とすると，

IS (o, p, s)
UNLESS (ENDS(o, p, Succ(s, a)))

IS (o, p, Succ(s, a))

という推論規則は，IS(o, p, s) が証明でき，EMDS(o, p, Succ(s, a))が証明できないならばIS(o, p, Succ(s, a)) が成立つ，ということを表現している．ENDS の定義により，先のカップと皿の移動の差を記述することができ

る．

UNLESS オペレータは Planner の THNOT と同じと考えてよく，知識の欠如を表す様相記号である．Prolog でも not があるので同様の記述が可能である．この方法は，しかしながら推論の量という点では，実際に公理を書き変える方法に劣る．

3. プランニング

2章で見たように知識の表現に用いられる枠組は様々であるが，これらのうちプランニングに適用できるのは論理だけである．フレーム理論や意味ネットワークは静的な知識しか表せないので当然としても，プロダクション・システムは手続きの記述が可能なのにもかかわらず，それらを主体としてアルゴリズムにまとめる方法が稀薄である．GPS (General Problem Solver)[8] のように理論としては存在しても，実際の複雑な問題に適用された例はないようである．本章では，実際のプランニングに用いられている言語として PLANNER と Prolog を取りあげる．

3.1 PLANNER

PLANNER[3] は Hewitt によりロボットのプランニングを行うための言語として提唱された．フル PLANNER はかなり大きな言語使用で，そのサブセットの Micro-PLANNER[12] が実際に作られた注2)．

結論からいうと，そのバックトラックの効率の悪さが批判され，ほとんど使われずに消えていった．SHRDLU が唯一の使用例ではないか．ただし数々の興味ある言語機能を持っており，その後のプログラミング言語に与えた影響は大きく，いまだに参照されることが多い言語である．

PLANNER の基本は一階述語論理であるが，一つの論理式をプログラムとして様々に用いることができるのを特徴とする．例えば

$\forall x$ human (x)→fallible (x)

という論理の式は，human ならば fallible である注3)．という知識を表してはいるが，その使い方を指示はしていない．PLANNER では，知識とその使い方は不可分であるとして，上記の式を以下の3通りに分類してプログラマが任意のものを選べるようにした．

① fallible (x) を証明したいときには human (x) を証明すればよい．

② human (x) が追加されたら fallible (x) も追加せよ．

③ fallible (x) が削除されたら human (x) も削除せよ．

PLANNER が普及しなかった最大の理由は，盲目的バックトラックの機能の効率の悪さである．例えば，ロボットが物を持つには右手と左手の両方が使える場合に，右手で持とうとしてもコップが熱過ぎてダメな場合に次に左手を試みるというような，原因を全く考慮しないバックトラックが起こってしまう．一般的には，例えば

(THAND (P) (Q) (R))

という3つのゴール注4)

を実行する場合に，(R) の失敗の原因が (P) にある場合でも，(Q) を何度もやり直すという点である．その後，この点をユーザが制御できるようにした CONNIVER[13] へと発展したが，こちらもあまり日の目を見ていない．

3.2 SHRDLU

Winograd の SHRDLU注5)[16] は総合的な知識表現，利用を伴う言語理解システムとして作られた．人間が言語を理解するときに，そのバックグラウンドの知識は不可欠である．SHRDLU ではすべての知識を手続き (PLANNER によるプログラム) として表現している．そのドメインとしては積木を操作するロボットの世界を扱い，人間は自然言語でロボットに命令を行う．ロボットには手と目があり，テーブル上の積木を操作したり，人間の質問に答えたり，あるいは人間から新しい情報を取り入れたりすることができる．SHRDLU の主目標は (1) 使いものになる言語理解システムを作ること，(2) 言語とは何であるか，どのようにして組み立てられているかを理解すること，(3) 知識とは何であるか，どのようにして計算機上で表現できるかを理解すること，の3つである．

SHRDLU では，したがって自然言語の構文解析から知識の表現，問題解決までが密接に関連したシステムとして構成されている．ここでは構文解析の部分は割愛し，残りの部分に関して紹介する．

SHRDLU では知識はすべてプログラムの形で表現される注6)．

例えば

I own red blocks.

という平叙文は

(THCONSE (X 1)

注2) Pico-PLANNER や Nano-PLANNER もある．

注3) 別の言い方をすれば

To err is human,

つまり，いつも過ちを犯すのは人間であって，コンピュータではないということ．

注4) PLANNER では，プリミティブには theorem の th をいつも頭に付け，THAND, THOR, THNOT, THASSERT のようになる．

注5) 英語におけるアルファベットの使用頻度が

ETAOINSHRDLU…

であることにちなんだ名前らしい．

注6) Winograd は知識の手続き的表現を提唱している[17]．

```
(#OWN : FRIEND $?X1)
  (THGOAL (#IS $?X1 #BLOCK)
  (THGOAL (#COLOR ?X1 #RED))
```

という Micro-PLANNER のプログラムに変換される．この意味は「ユーザ（：FRIEND）がなにかを所有していることを証明したいときは，いつでもそれがブロックであり，かつ赤いことを証明することによって行うことができる」という意味である．

単純にいうと，平叙文は定理の形，命令文は定理証明の形（THGOAL），質問文は変数を含む定理証明（THFIND）の形に変換される．

SHRDLU の積木の世界を紹介しよう．世界にはロボット（：SHRDLU）と人間（：FRIEND）がおり，ロボットには手（：HAND）があって，テーブル（：TABLE）の上の物体を操作することができる．テーブルの上には1つの箱（：BOX）と，いくつかの物体（：B1, ：B2, …）がある．また，これらの物体やその属性を記述するのに用いる概念の集りは図3のようになっている．ここで，#PHYSOB は物体を，#MANIP はロボットの持ち上げることのできるものを表す．

これらを用いて，(#IS :SHRDLU #ROBOT), (#IS :B5 #PYRAMID), (#PYSOB : TABLE) のような主張をする．図3の木の末端にあるものは個体として扱われ #IS の引数となり，それ以外はそれ自身が述語として扱われている．ただし，#PROPERTY は別で，(#COLOR :BOX #WHITE) のように用いられる．また，物体の大きさや位置は (#SIZE :B5 (100 100 300)), (#AT : B5 (400 600 200)) のように表現される．

この世界のなかでロボットが動作するわけだが，基本動作としては MOVETO と GRASP, UNGRASP の3種類だけである．SHRDLU では，これらは表示ルーチンに送られるだけであるが，実際のロボットへの指令として考えてもよい．これらの基本動作の組み合わせで，ゴールを達成することができる．PLANNER は例えば (THGOAL (#GRASP :B1) (THUSE TC-GRASP)) のようなゴールを達成しようと試みることになる．この TC-GRASP という定理は

```
(DEFTHEOREM TC-GRASP
  (THCONSE (XY) (#GRASP $?X)
    (THGOAL (#MANIP $?X))
    (THCOND ((THGOAL (#GRASPING $?X)))
      ((THGOAL (#GRASPING $Y))
       (THGOAL (#GET-RID-OF $?Y)
           (THUSE TC-GET-RID-OF))))
      (T))
  (THGOAL (#CLEARTOP $?X)
```

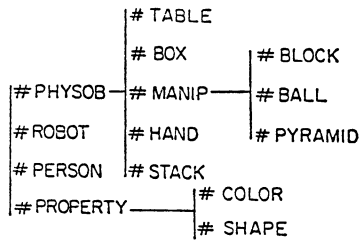


図3 SHRDLU における概念の構造

```
  (THUSE TC-CLEARTOP))
(THSETQ $Y (TOPCENTER $?X)
(THGOAL (#MOVEHAND $?Y)
  (THUSE TC-MOVEHAND))
(THASSERT (#GRASPING $?X))))
```

のように記述されており，さらに細かいゴールの列に展開される．

目的のゴールに到達するには筋道が決まっているとは限らない．このような場合には，PLANNER の持つバックトラックの機能を用いて正しいプランが生成されるまで探索が行われる．この際，例えば #MOVEHAND は実際に：HAND を動かすのではなく，実際の移動の計画を立てるだけである．計画が完成して始めて実行に移される．

3.3 Prolog

Prolog[15,18,19] はフランスのマルセーユ大学の Colmerauer らによって 1972 年に作られたプログラミング言語である[1]．

Prolog で基本となる述語というのは主語・述語の述語で，

私は買物に行く．

という文の「行く」というところがそうである．この述語を中心にして，Prolog ではこの文を

```
go(i, shopping).
```

のように表現する．このように，日常我々が用いている言語に近い表現が可能であるのが Prolog の特徴の一つである．

Prolog の文には大きく分けて3つの形がある注7)．

```
father(cain, adam).
```

(cain の father は adam である）のように単純な事実を表すもの，

```
?-father(juliet, X).
```

(juliet の father は誰か）のようにシステムに質問を発

注7) Prolog には様々の方言があるだけで，いまだに標準語というものがない．ここでは，今のところ一番勢力を持っているエジンバラ Prolog[9] のシンタックスを使うことにする．シンタックスは Prolog の本質とはあまり関係がない．

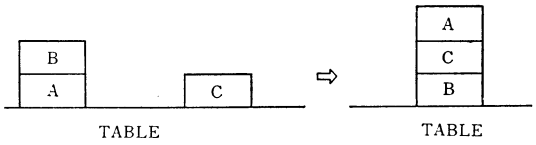


図 4 積 木 の 問 題

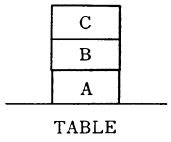


図 6 図 4 で on(c, b) を先に解いた状態

するもの,

grand_father (X, Y) : -
parent(X, Z), father(Z, Y).

(Xの grand_father は, Xの parent Z の father である) のように新しい規則を与えるもの, の 3 つである. これらの例で 大文字で 始まっている ものが 変数である (この制約のために adam, juliet などと全部小文字にしていた). 質問を発した場合には 変数の値を答えてくれる. また, 最後の例は : - の右側が成り立っているときに, : - の左側も成り立つという規則で, 一般に

A : -B, C, D.

は, BかつCかつDならばAという意味になる. このような規則を書き並べておくと, 後はシステムが勝手に推論をやってくれる.

同じ名前の事実や規則が 2 つ以上あってもかまわない. 例えば parent というのは father か mother のことであるから

parent(X, Y) : -father(X, Y).
parent(X, Y) : -mother(X, Y).

となる. システムが場合に応じてそれらのうちから適当なものを捜して使ってくれるわけだが, そのときに動くのがユニフィケーションとバックトラックの機能である.

ユニフィケーションというのは一種のパターン・マッチングのことであるが, PLANNER のそれと根本的に異なるのは, 呼出し側と規則の側の両方に変数が存在しうる点である. PLANNER のそれは, 情報が呼出し側から呼ばれた側への 一方通行であったが, Prolog の場合は両方向に情報が流れるので, この機能を用いてより柔軟なプログラミングが可能になっている.

3.4 WARPLAN

Prolog を用いたプランニングシステムとして WARPLAN[14] がある. これは, 例えば積木の世界などで図 4 の状態から

on(a, c), on(c, b)

のような複数のゴールを同時に解決することの可能なシステムである. 図 4 の問題は一見簡単そうであるが, 例えば

on(a, c)

を先に解くと図 5 の状態になるし,

on(c, b)

を先に解くと図 6 のようになってしまう. つまり両者をうまく織り混ぜて同時に解かなければならないのである.

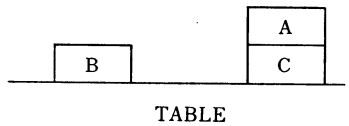


図 5 図 4 で on(a, c) を先に解いた状態

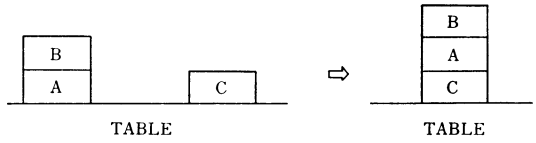


図 7 WARPLAN で解けない積木の問題

WARPLAN ではまず一方の問題を解き, もう一方の解をその間にうまく挿入するという方法を取っている. 図 4 の場合には, 例えば

on(a, c)

を解く動作の列

move(b, a, table) ; move(a, table, c)

の間に

move(c, table, b)

を挿入すればよい.

しかし, この方法にも限界があり, 例えば図 7 の問題は解けない. 一方のゴール

on(b, a)

が既に満たされているにもかかわらず, この問題を解くにはこの状態を一度壊すという遠回りをしなければならないからである. このように, 一般にプランニングシステムにとっては, 遠回りというのは最も苦手の問題である.

4. ま と め

知識とプランニングとを有機的に結びつけた説明を行いたかったのだが, それには見事に失敗してしまった. SHRDLU のように, かつてそれらが同時に研究された時期があるが, その後問題が細分化され, 知識表現はそれ自身で, またプランニングはそれ自身で, それぞれ固有の問題を抱えている現状では, 両者はそれぞれ別々に研究されている. ここでは, 紙面の都合もあって知識表現に重点を置いた. プランニングに関してもいろいろと話題はあって, 特にいくつかのゴールを同時に満足させ

なければならない状態でのプランニングは WARPLAN 以外にもいろいろと研究されている．しかしこれに関しては別の機会を持ちたい．

参考文献

1) G. Battani and H. Meloni, "Interpreteur du Langage de Programmation Prolog", Groupe d'Intelligence Artificielle, Mareseille-Luminy, 1973
2) R. E. Fikes and N. J. Nilsson, "STRIPS : A new Approach to the Application of Theorem Proving to Problem Solving", Artificial Intelligence 2, pp. 189-208, 1971
3) Carl Hewitt, "PLANNER : A Language for Proving Theorems in Robot" Proc. of IJCAI-I, pp. 295-301, 1969
4) Robert A. Kowalski, "Predicate Logic as Programming Language" Proc. of IFIP Congress, 1974
5) John McCarthy and P. J. Hayes, "Some Philosophical Problems from the Standpoint of Artificial Intelligence", Machine Intelligence 4 (eds. B. Meltzer and D. Michie), pp. 463-502, Edinburgh University Press, 1969
6) Marvin Minsky, "A Framework for Representing Knowledge", in The Psychology of Computer Vision, McGraw Hill, 1975
7) Hideyuki Nakashima, "Prolog/KR-Language Features", Proc. of the First International Logic Programming Conference, pp. 65-70, 1982
8) Allen Newell and Herbert A. Simon, "Human Problem Solving", Prentice Hall Inc., New Jersey, 1972
9) L. M. Pereira, F. C. N. Pereira and D. H. D. Warren, "User's guide to DECsystem-10 PROLOG", Dept. of Artificial Intelligence, Edinburgh, 1978
10) M. R. Quillian, "Semantic Memory" in Semantic Information Processing (M. Minsky ed.), MIT Press, 1968
11) E. Sandewall, "An Approach to the Frame Problem and its Implementation", Machine Intelligence 7, Edinburgh University Press, pp. 195-204, 1972
12) Gerald J. Sussman, T. Winograd and E. Charniak, "MICRO-PLANNER Reference Manual", MIT AI Memo No. 203A, 1971
13) G. J. Sussman and D. McDermott, "From Planner to Conniver : a Genetic Approach", Proc. of FJCC 41, AFIP, 1972
14) D. H. D. Warren, "WARPLAN : A System for Generating Plans", Dept. of Computation logic Memo No. 76, Edinburgh, 1974
15) D. H. D. Warren, M. L. Pereira and F. C. N. Pereira, "Prolog-the Language and its Implementation Compared with Lisp", Proc. of the Symposium on Artificial Intelligence and Programming Language, ACM SIGPLAN/SIGART, 1977
16) Terry Winograd, "Understanding Natural Language", Academic Press, 1972
淵 一博, 田村浩一郎, 白井良明訳："言語理解の構造", 産業図書, 1976
17) Terry Winograd, "Frames and Declarative-Procedural Controversy", in Representation and Understanding, Academic Press, 1975
18) 中島秀之, "Prolog", 産業図書, 1983
19) 後藤滋樹, "PROLOG 入門", サイエンス社, 1984



中島秀之

(Hideyuki NAKASHIMA)

昭和27年生れ．52年東京大学工学部計数工学科卒業．53～54年 MIT 留学．58年東京大学大学院情報工学専門課程修了．工学博士．現在，電総研人間機械システム研究室．人工知能に興味を持つ．情報処理学会，日本認知科学会，日本ソフトウェア科学会，ACM 各会員．